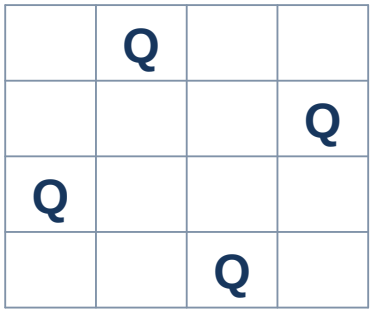
# N-QUEEN PROBLEM IN AI

Detailed Theory, Solved Numericals, Algorithms and Python Programs

**Study Notes / Assignment / Practical File**




One valid 4-Queen arrangement: rows 1–4 use columns 2, 4, 1, 3.

Contents include definitions, AI formulation, constraints, backtracking, multiple worked examples, pseudocode, Python implementations, sample outputs, complexity, and viva questions.

# 1. Introduction

Artificial Intelligence (AI) is a branch of computer science concerned with building systems that perform tasks associated with intelligent behaviour, such as reasoning, searching, planning, and problem-solving. Many AI problems are represented as a collection of possible states and actions. A search algorithm explores these states to find a state that satisfies the goal.

The N-Queen Problem is a well-known AI search problem. It is used to demonstrate constraint satisfaction, recursive search, and backtracking.

# 2. Definition of the N-Queen Problem

The **N-Queen Problem** is the task of placing **N chess queens on an N × N chessboard** so that no two queens attack each other. A queen can attack another piece horizontally, vertically, or diagonally. Therefore, no two queens in a valid solution may share a row, a column, or a diagonal.

**Example:** The 4-Queen problem asks us to place four queens on a 4 × 4 chessboard without any pair of queens attacking one another.

**Goal:** Find one valid arrangement or list every valid arrangement for the given value of N.

# 3. Important Definitions

**Chessboard:** A square board with N rows and N columns, giving $N^2$ possible squares.

**Queen:** A chess piece that can move any number of squares horizontally, vertically, or diagonally, provided its path is unobstructed.

**State:** A partial or complete arrangement of queens on the board.

**Initial state:** The empty chessboard before any queens have been placed.

**Goal state:** A complete arrangement containing N queens in which all constraints are satisfied.

**Constraint:** A rule that restricts which choices are allowed. Here, queens cannot share a row, column, or diagonal.

**Search space:** The set of possible states or arrangements considered by the algorithm.

**Constraint Satisfaction Problem (CSP):** A problem defined by variables, domains of possible values, and constraints that values must satisfy.

**Backtracking:** A search method that makes a choice, explores its consequences, and undoes the choice when it cannot lead to a solution.

**Recursion:** A programming technique in which a function calls itself to solve a smaller part of a problem.

**Pruning:** Rejecting a partial solution as soon as it violates a constraint, rather than exploring it further.

**Solution:** A complete assignment of queens that satisfies every rule.

# 4. Rules and Mathematical Constraints

A standard solution places one queen in each row. This automatically prevents row conflicts. The algorithm must then check columns and diagonals.

**Rule 1 — Row:** Each row contains exactly one queen.

**Rule 2 — Column:** No two queens may be in the same column.

**Rule 3 — Diagonal:** No two queens may share a diagonal.

If two queens are located at (r■, c■) and (r■, c■), they share a diagonal exactly when **|r■ – r■| = |c■ – c■|**. They share a column when c■ = c■.

For example, (1, 2) and (3, 4) are diagonal positions because |1 – 3| = |2 – 4| = 2. They cannot both contain queens.

# 5. AI Formulation as a Constraint Satisfaction Problem

**Variables:** Q■, Q■, …, Q■, where Q■ represents the column selected for the queen in row r.

**Domain:** Each variable can take a column value from 1 to N.

**Constraints:** For every pair of different rows i and j, Q■ ≠ Q■ and |Q■ – Q■| ≠ |i – j|.

**Goal test:** All N variables are assigned values and all constraints hold.

This formulation is compact: instead of representing every square independently, it stores only one column number per row.

## 6. Backtracking Technique in Detail

Backtracking constructs a solution incrementally. It places a queen in the current row, checks whether the position is safe, and then recursively handles the next row. If no safe column remains, it removes the previously placed queen and tries a different column.

**Working steps:**

1. Start with an empty board.
2. Begin at the first row.
3. Try each column in the current row.
4. Check whether the position conflicts with any previously placed queen.
5. If safe, place the queen and move to the next row.
6. If all N rows are filled, record a solution.
7. If a row has no safe position, undo the last placement and try another choice.
8. Continue until one solution is found or every possibility has been checked.

**Why it is useful:** A brute-force method could examine enormous numbers of complete boards. Backtracking prunes a branch as soon as a conflict is found.

## 7. Numerical Example 1: N = 1

A 1 × 1 board has one square. Place the only queen in that square.

```
Board:
Q

Position: (row 1, column 1)
Number of solutions: 1
```

There are no other queens, so no conflict is possible. Therefore, N = 1 has one solution.

## 8. Numerical Example 2: N = 2

Try to place two queens on a 2 × 2 board. If the first queen is placed in row 1, column 1, then the queen in row 2 cannot use column 1 (same column) or column 2 (diagonal). No safe square remains.

```
Q .
. Q    -> diagonal conflict

Q .
Q .    -> same-column conflict
```

By symmetry, starting in column 2 leads to the same situation. Therefore, **the 2-Queen problem has no solution**.

## 9. Numerical Example 3: N = 3

For a 3 × 3 board, try placing a queen in the first row. The next rows are restricted by the column and diagonal rules. Every possible branch eventually creates a conflict, so no complete arrangement is possible.

```
Possible conflict patterns (illustrative):

Q . .       . Q .
. . Q       Q . .
. Q .       . . Q

In each shown pattern, at least two queens share a diagonal.
```

After backtracking through the possible first-row choices, no valid board is found. Therefore, **the 3-Queen problem has no solution**.

## 10. Numerical Example 4: N = 4 (Solved)

For a 4 × 4 board, one solution is the column sequence **[2, 4, 1, 3]**. This means the queen positions are (1,2), (2,4), (3,1), and (4,3).

| Row / Col | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | · | Q | · | · |
| 2 | · | · | · | Q |
| 3 | Q | · | · | · |
| 4 | · | · | Q | · |

### Step-by-step verification

• **Rows:** One queen appears in every row.
• **Columns:** 2, 4, 1, and 3 are all different.
• **Diagonals:** For every pair of queens, the row difference is not equal to the column difference in absolute value.

Examples: Between (1,2) and (2,4), row difference = 1 and column difference = 2, so they are not diagonal. Between (1,2) and (3,1), the differences are 2 and 1, so they are not diagonal. The same test can be applied to all remaining pairs.

**Another solution:** [3, 1, 4, 2]. Thus, there are 2 distinct 4-Queen solutions when rotations and reflections are counted as different arrangements.

## 11. Numerical Example 5: N = 5

One valid solution for N = 5 is **[1, 3, 5, 2, 4]**. The numbers give the queen's column in rows 1 through 5.

| Row / Col | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | Q | · | · | · | · |
| 2 | · | · | Q | · | · |
| 3 | · | · | · | · | Q |
| 4 | · | Q | · | · | · |
| 5 | · | · | · | Q | · |

Verification: columns 1, 3, 5, 2, 4 are unique. For every pair of rows, the absolute row difference differs from the absolute column difference, so there are no diagonal conflicts.

This example demonstrates that the method works for larger boards as well. The program can search for solutions for other positive values of N.

## 12. Pseudocode

```
N_QUEENS(row):
    if row == N:
        save current arrangement
        return

    for col from 0 to N - 1:
        if (row, col) is safe:
            place queen at (row, col)
            N_QUEENS(row + 1)
            remove queen at (row, col)  # backtrack
```

The base case is reached when row == N, meaning every row has a queen. The recursive case tests columns, makes a safe placement, and searches the next row.

## 13. Python Program 1: Find All Solutions

This program uses a list where board[row] stores the column containing the queen. Python indexes begin at 0, so a position (0, 1) corresponds to row 1, column 2 in normal board notation.

```
def solve_n_queens(n):
    solutions = []
    board = [-1] * n

    def is_safe(row, col):
        for old_row in range(row):
            old_col = board[old_row]

            # Same column
            if old_col == col:
                return False

            # Same diagonal
```

```
            if abs(old_col - col) == abs(old_row - row):
                return False

        return True

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return

        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1  # undo placement

    backtrack(0)
    return solutions


n = int(input("Enter N: "))

if n <= 0:
    print("Enter a positive integer.")
else:
    solutions = solve_n_queens(n)
    print("Number of solutions:", len(solutions))

    for number, solution in enumerate(solutions, start=1):
        print("\nSolution", number)
        for queen_col in solution:
            line = ""
            for col in range(n):
                line += "Q " if col == queen_col else ". "
            print(line)
```

## 14. Sample Output for Python Program 1

```
Enter N: 4
Number of solutions: 2

Solution 1
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2
. . Q .
Q . . .
. . . Q
. Q . .
```

The output shows two solutions. Q represents a queen, and a dot represents an empty square. The order of solutions may differ if the column search order changes.

## 15. Python Program 2: Find Only One Solution

If only one answer is required, the program can stop as soon as a complete arrangement is found.

```
def one_solution(n):
    board = [-1] * n

    def safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col or abs(c - col) == abs(r - row):
                return False
        return True

    def place(row):
        if row == n:
            return True

        for col in range(n):
            if safe(row, col):
                board[row] = col
                if place(row + 1):
                    return True
                board[row] = -1
        return False

    if place(0):
```

```
        return board
    return None


n = int(input("Enter N: "))
if n <= 0:
    print("Enter a positive integer.")
else:
    answer = one_solution(n)
    if answer is None:
        print("No solution exists.")
    else:
        for queen_col in answer:
            print(" ".join(
                "Q" if col == queen_col else "."
                for col in range(n)
            ))
```

**Difference:** Program 1 explores all branches to collect every solution. Program 2 returns immediately when it finds its first complete solution.

# 16. Python Program 3: Count Solutions Only

This version counts complete arrangements without storing each board.

```
def count_n_queens(n):
    board = [-1] * n
    count = 0

    def safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col or abs(c - col) == abs(r - row):
                return False
        return True

    def backtrack(row):
        nonlocal count
        if row == n:
            count += 1
            return

        for col in range(n):
            if safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1

    backtrack(0)
    return count


n = int(input("Enter N: "))
if n <= 0:
    print("Enter a positive integer.")
else:
    print("Total solutions:", count_n_queens(n))
```

For N = 4, this program prints **Total solutions: 2**. It is useful when the number of solutions is needed but printing every board is unnecessary.

# 17. Dry Run for N = 4

The following table gives a simplified illustration of the recursive search. A safe choice is followed by a deeper search; if the next row cannot be completed, the algorithm returns to the earlier row.

| Stage | Action | Explanation |
|---|---|---|
| 1 | Start at row 0 | Board is empty. |
| 2 | Try a column | Check column and diagonal constraints. |
| 3 | Place queen if safe | Move to the next row recursively. |
| 4 | Reach a dead end | If no column is safe, undo the previous placement. |
| 5 | Try another column | Continue exploring alternatives. |
| 6 | Reach row N | A complete valid arrangement is recorded. |

The exact sequence depends on the order in which columns are tried. The algorithm's correctness does not depend on that order.

## 18. Complexity Analysis

**Time complexity:** The backtracking search is often described with an approximate worst-case bound of O(N!) because it assigns queens to distinct columns row by row. The safety check itself can take O(N), so a straightforward implementation may incur additional work per attempted placement. Actual runtime depends heavily on pruning and N.

**Auxiliary space:** O(N) for the board list and recursion depth, excluding stored solutions.

**Space when storing all solutions:** Additional memory is proportional to the number of solutions multiplied by N if each solution is stored as N column values.

## 19. Advantages

• Easy to understand and implement.
• Rejects invalid partial arrangements early.
• Demonstrates recursion and systematic search.
• Can be used to find one solution, all solutions, or only the count.
• Provides a practical introduction to constraint-satisfaction problems.

## 20. Limitations

• Search time grows quickly as N increases.
• Listing every solution may require substantial time and memory.
• Basic backtracking uses simple variable and value ordering; larger variants can benefit from heuristics and symmetry reduction.

## 21. Applications and Learning Value

The N-Queen Problem is mainly an educational example. Its ideas are related to scheduling, resource allocation, map colouring, and other tasks in which choices must satisfy multiple constraints. It demonstrates how a problem can be represented formally and solved through a search strategy.

## 22. Common Errors in Python

• **Forgetting to backtrack:** Reset the queen position after a recursive call so the next candidate can be tried.
• **Checking only columns:** Diagonal conflicts must also be checked.
• **Using the wrong diagonal formula:** Use abs(old_col - col) == abs(old_row - row).
• **Not copying a solution:** Append board.copy(), not board, because the board changes during later recursion.
• **Confusing indexes:** Python rows and columns start at 0; human-readable chessboard positions often start at 1.

## 23. Quick Revision

**Definition:** Place N queens on an N × N board so no two attack each other.
**Technique:** Backtracking.
**Constraints:** No shared row, column, or diagonal.
**Representation:** One column value per row.
**Safe diagonal condition:** |r■ – r■| ≠ |c■ – c■| for every pair of queens.
**N = 1:** 1 solution.
**N = 2:** 0 solutions.
**N = 3:** 0 solutions.
**N = 4:** 2 solutions.

## 24. Viva Questions with Answers

**1. What is the N-Queen Problem?**
A problem of placing N queens on an N × N board so no two queens attack each other.

**2. Why is it used in AI?**
It demonstrates search, constraint satisfaction, and backtracking.

**3. What is backtracking?**
A method that undoes a choice when it cannot lead to a solution and tries another choice.

**4. Why is one queen placed per row?**
It enforces the row constraint automatically and simplifies the search.

**5. How are diagonal conflicts detected?**
Two queens are diagonal when their absolute row difference equals their absolute column difference.

**6. Why is board.copy() used?**
To save a snapshot of the current solution rather than a reference to the changing list.

**7. Does N = 2 have a solution?**
No.

**8. How many solutions does N = 4 have?**
Two, counting rotations and reflections as distinct arrangements.

**9. What is pruning?**
Stopping exploration of a partial arrangement as soon as it violates a constraint.

**10. Can the program find all solutions?**
Yes, by continuing the search after recording each complete arrangement.

## 25. Conclusion

The N-Queen Problem is a classic AI constraint-satisfaction problem. Backtracking solves it by placing queens one row at a time, testing column and diagonal constraints, and undoing choices when a branch fails. The numerical examples for N = 1, 2, 3, 4, and 5 and the three Python programs show how the method works in practice.